

Core Concepts — Diffusion and the Generative Landscape

Part 05 — 🌈 Generative AI · 05-genai/notes/01-core-concepts.md

Modules 03-04 taught you a generative model without calling it that: a language model learns $P(\text{next token} \mid \text{context})$ and samples from it. This module widens the lens to pixels, waveforms, and video, and spends most of its time on the family that now dominates all three: **diffusion**.

1. Generative vs. discriminative

A **discriminative** model learns $P(y|x)$ — draw the boundary. A **generative** model learns (or can sample from) $P(x)$ itself. An image is a point in a $\sim$ million-dimensional space (pixels x channels); almost all of that space is static, not images. The generative model's job is learning the thin, curved manifold where real images live, and how to land on it.

The family tree

Family	Core idea	Strength	Fatal-ish weakness
Autoregressive (PixelCNN; text LLMs)	factorise $P(x) = \prod P(x_i \mid x_{<i})$, generate one unit at a time	exact likelihood, stable training	sequential generation is painfully slow at pixel resolution
VAE (Module 02 §5.5)	encode to a probabilistic bottleneck, decode back	fast single-pass sampling, usable latent space	blurry — the ELBO objective averages away uncertainty
GAN	generator vs. discriminator, adversarial training	sharp, one-pass sampling	unstable training, mode collapse , no likelihood, hard to condition
Diffusion	learn to reverse a gradual noising process	sharp, diverse, stable to train, easy to condition	slow sampling (many steps)
Normalizing flows	invertible transforms, tractable Jacobian	exact likelihood and inversion	invertibility constrains architecture; hasn't scaled as well

Why diffusion won

GAN training is a minimax game with no convergence guarantee: mode collapse (the generator finds a few outputs that reliably fool the discriminator and stops exploring), instability that needs babysitting, and no likelihood to lean on for conditioning or evaluation. VAEs train stably but the objective rewards hedging — unsure whether an edge is sharp-A or sharp-B, the decoder averages them, hence the blur.

Diffusion training is **plain regression** (predict the noise you added — no adversary, no minimax, no mode collapse), which is exactly why it scaled to internet-size datasets without a team babysitting instability. The price is sampling cost: instead of one forward pass you run the network tens to hundreds of times. Nearly the

entire post-2020 diffusion literature (DDIM, distillation, consistency models, flow matching) is attacking that one price tag.

Opinion: if you're generating images or audio today, you're using diffusion or a close cousin (flow matching, which trained Stable Diffusion 3 and Flux) unless you have a specific reason not to. GANs still win a narrow set of real-time, single-domain tasks (face reenactment, some super-resolution) — as a default, that era is over.

2. Diffusion, in full

One sentence: teach a network to remove noise, and it learns to draw.

2.1 Forward process — destroy the image on a schedule

Take a real image x_0 , add a little Gaussian noise, repeat T times ($T=1000$ in the original DDPM). After enough steps x_T is indistinguishable from pure noise regardless of x_0 . No learning required — it's arithmetic, governed by a variance schedule β_t :

$$x_t = \sqrt{1-\beta_t} \cdot x_{t-1} + \sqrt{\beta_t} \cdot \epsilon, \quad \epsilon \sim N(0, I)$$

Because Gaussians compose, you can jump straight from x_0 to any x_t in closed form:

$$x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{1-\alpha_t} \cdot \epsilon, \quad \alpha_t = \prod_{s \leq t} (1-\beta_s)$$

This closed-form $q(x_t|x_0)$ is the most important equation here: training needs no simulation loop — pick a random x_0 , a random t , one ϵ , compute x_t in one line, and you have a training example.

2.2 Reverse process — learn to undo it

You can't analytically invert the forward process (that requires $P(x_0)$ — the whole problem). Instead train a network $\epsilon_\theta(x_t, t)$ to **predict the noise that was added**:

$$L(\theta) = E_{\{x_0, t, \epsilon\}} [|| \epsilon - \epsilon_\theta(x_t, t) ||^2]$$

Plain MSE, no adversary, no bound to fuss over — the same supervised training loop from Module 01, with a random timestep as an extra input. Generation runs it backward: start from pure noise $x_T \sim N(0, I)$ and iteratively subtract the estimated noise, a little at a time, until you land on a clean x_0 . "A little at a time" matters: ϵ_θ predicts *aggregate* noise from 0 to t , not the individual increment, so subtracting it all in one leap produces a blurry average over every image consistent with that noisy input — the same averaging failure that blurs VAEs. Small steps let the model repeatedly re-commit to sharper detail as the signal firms up; that iterative refinement is the actual source of diffusion's sample quality.

2.3 DDPM vs. DDIM sampling

DDPM sampling is stochastic: each reverse step subtracts the predicted noise *and* injects fresh, smaller noise back in, matching the process the model was derived to reverse. That fidelity costs steps — skip them and the injected noise no longer matches the schedule, so DDPM wants most of T (hundreds+) for good samples.

DDIM re-derives the reverse process as a non-Markovian, **deterministic** mapping trained with the exact same objective but not required to visit every step. Practically: 20-50 steps for quality comparable to 1000-step DDPM, and — because it's deterministic — the same seed always reproduces the same image, and two seeds' latents interpolate meaningfully. That determinism is why DDIM-family samplers became the default in production tools: users expect "same seed, same image" and expect seconds, not minutes.

	DDPM	DDIM
Reverse step	stochastic	deterministic (or tunable via η)
Steps for good quality	~1000	20-50
Reproducible from seed	no	yes
Latent interpolation	not meaningful	meaningful

Later samplers (DPM-Solver, Euler ancestral, UniPC — the dropdown in ComfyUI/A1111) are higher-order numerical solvers for the same underlying ODE/SDE DDIM formalized. Same story: fewer steps, comparable quality, more implementation complexity. `code/guidance_and_sampling.py` demonstrates the steps-vs-quality trade-off numerically.

2.4 Classifier-free guidance

Conditioning the denoiser directly, $\epsilon_{\theta}(x_t, t, c)$, works but tends to be "soft" — the model listens to the prompt *and* to its generic sense of images, diluting c 's influence.

Classifier-free guidance (Ho & Salimans, 2022) fixes this with no separate classifier. During training, randomly drop the condition (train ~10% of examples with $c = \emptyset$). At sampling time run the network twice per step — conditioned and unconditioned — and extrapolate away from the unconditional prediction:

$$\epsilon_{\text{guided}} = \epsilon_{\theta}(x_t, t, \emptyset) + w \cdot (\epsilon_{\theta}(x_t, t, c) - \epsilon_{\theta}(x_t, t, \emptyset))$$

w is the **guidance scale**. $w=1$ is plain conditional generation; $w=0$ ignores the prompt; $w>1$ (typically 5-15) *exaggerates* the prompt's push — more literal, but also more saturated and eventually artifacted. There is no free lunch:

Guidance scale w	Prompt fidelity	Diversity across seeds	Artifacts
~1	weak	high	soft, generic
~5-8 (typical default)	good	moderate	rare
~15-20	very literal	low — samples converge	oversaturated
~30+	prompt-obsessed	collapsed	severe

`code/guidance_and_sampling.py` sweeps this on a toy 2D conditional distribution so you see the fidelity/diversity trade-off in numbers, not on faith.

2.5 Latent diffusion — why Stable Diffusion isn't pixel-space

Diffusing directly on a 512x512x3 grid means every one of ~50 reverse steps needs a full network pass over ~750K numbers. **Latent diffusion** (Rombach et al., 2022 — this paper is Stable Diffusion) trains a VAE to compress images into a much smaller latent grid — e.g. 64x64x4, a **48x** reduction — and runs the entire diffusion process there. Only at the end does the VAE decoder turn the final latent back into pixels.

```
image(512x512x3) --[VAE encoder]--> z(64x64x4) --[diffusion U-Net/DiT]--> z' --[VAE decoder]--> image
```

This is why Stable Diffusion runs on a consumer GPU while pixel-space DDPM topped out around 256x256 on serious hardware: the VAE buys roughly an order of magnitude in compute for a small, usually imperceptible quality cost. That VAE reconstruction error is exactly why fine, high-frequency detail — text, hands — is where latent diffusion's seams show (more in note 02).

3. The architecture shift: U-Net → Diffusion Transformer

The original denoiser was a **U-Net**: a convolutional encoder downsampling through resolutions to a bottleneck, then a decoder upsampling back, with skip connections carrying fine spatial detail around the bottleneck. Timestep and text conditioning enter via added embeddings and **cross-attention** spliced into the U-Net's blocks.

Starting 2022-2023 the field did to diffusion what Module 03 did to NLP: swapped the backbone for a transformer. The **Diffusion Transformer (DiT)** (Peebles & Xie, 2022; scaled up in Sora and Stable Diffusion 3/Flux) treats the latent grid as a sequence of patches — exactly a Vision Transformer — and runs standard self-attention + MLP blocks over them, injecting timestep/conditioning via adaptive layer norm instead of U-Net's skip connections. Same reasons transformers took over NLP:

- **Scaling behaviour.** U-Net's inductive biases (locality, translation equivariance) help at small scale and become a ceiling at large scale — the CNN-vs-transformer story in vision classification, again. DiT follows transformer-style scaling-law curves (Module 04's territory), a different modality.
- **Shared infrastructure.** A DiT and an LLM share attention/FFN building blocks, so kernel and parallelism investment transfers directly.
- **Conditioning as tokens.** Attention already handled cross-modal conditioning; a transformer backbone makes text, image, and (for video) time all first-class tokens in one sequence instead of bolted-on side channels.

	U-Net	DiT
Core op	convolution + skip connections	self-attention over patch tokens
Inductive bias	strong	weak — learns structure from data
Scaling behaviour	plateaus	follows transformer scaling laws
Conditioning	cross-attention spliced in	native, tokens throughout
Used by	SD 1.x/2.x, most 2023-era fine-tunes	Sora, SD3, Flux, most 2024+ frontier systems

4. Forward and backward links

Idea here	Where it connects
VAEs, latent space	Module 02 §5.5, §6 — the direct ancestor of latent diffusion
Cross-attention	Module 03 — the same mechanism, doing double duty
Scaling laws behind DiT	Module 04 — same curves, different modality
Text-to-image pipeline, video, audio, multimodal, evaluation, failure modes	notes/02-multimodal-and-practice.md [05-genai/notes/02-multimodal-and-practice.md]